OXFORD CDE

Journal of Computational Design and Engineering, 2021, 9(1), 114–127

https://doi.org/10.1093/jcde/qwab072
Journal homepage: www.jcde.org

RESEARCH ARTICLE

Dataset and method for deep learning-based reconstruction of 3D CAD models containing machining features for mechanical parts

Hyunoh Lee[1], Jinwon Lee[1], Hyungki Kim[2] and Duhwan Mun[1,*]

[1]School of Mechanical Engineering, Korea University, 145 Anam-ro, Seongbuk-gu, Seoul 02841, Republic of Korea and [2]Division of Computer Science and Engineering, Jeonbuk National University, 567, Baekje-daero, Deokjin-gu, Jeonju, Jeollabuk-do 54896, Republic of Korea

*Corresponding author. E-mail: dhmun@korea.ac.kr https://orcid.org/0000-0002-5477-0671

Abstract

Three-dimensional (3D) computer-aided design (CAD) model reconstruction techniques are used for numerous purposes across various industries, including free-viewpoint video reconstruction, robotic mapping, tomographic reconstruction, 3D object recognition, and reverse engineering. With the development of deep learning techniques, researchers are investigating the reconstruction of 3D CAD models using learning-based methods. Therefore, we proposed a method to effectively reconstruct 3D CAD models containing machining features into 3D voxels through a 3D encoder–decoder network. 3D CAD model datasets were built to train the 3D CAD model reconstruction network. For this purpose, large-scale 3D CAD models containing machining features were generated through parametric modeling and then converted into a 3D voxel format to build the training datasets. The encoder–decoder network was then trained using these training datasets. Finally, the performance of the trained network was evaluated through 3D reconstruction experiments on numerous test parts, which demonstrated a high reconstruction performance with an error rate of approximately 1%.

Keywords: 3D CAD model; 3D reconstruction; convolutional neural networks; encoder and decoder; deep learning; machining features

1. Introduction

Amidst the development of three-dimensional (3D) computer-aided design (CAD) and computer-aided manufacturing, various 3D CAD model reconstruction techniques from objects such as 2D drawings, stereo images, and scanned point clouds are being developed. Furthermore, these techniques are used for numerous purposes across various industries, including free-viewpoint video reconstruction, robotic mapping, tomographic reconstruction, 3D object recognition, and reverse engineering.

Traditional 3D reconstruction methods do not show good reconstruction performance if the required information and input data are insufficient or if the input data contain noise. For example, when reconstructing a 3D model from a single image, prior knowledge and assumptions are necessary (Fu *et al.*, 2021). Besides, the reconstruction target is limited to a specific category. If the conditions are not satisfied, a model will not be fully reconstructed. Moreover, the reconstruction performance decreases if the image data or scanned point clouds contain bias or occlusion (Eltner *et al.*, 2016).

Fueled by the recent rapid advancements in deep learning techniques, researchers in the 3D CAD model reconstruction field have made various attempts to reconstruct 3D models by

Received: 3 May 2021; **Revised:** 28 October 2021; **Accepted:** 3 November 2021

applying the latest networks, such as 3D convolutional neural networks (CNNs). Related studies include methods to reconstruct 3D shapes from a single image or multiple images (Wu *et al.*, 2016; Tulsiani *et al.*, 2017; Xu *et al.*, 2019) and reconstruct 3D shapes from point clouds or 3D voxels (Sharma *et al.*, 2016; Dai *et al.*, 2017b; Ge *et al.*, 2018; Chen *et al.*, 2019). As a 3D data format, 3D voxel models with a predetermined resolution are often used because the size of a 3D voxel model is fixed; this characteristic of the voxel data structure is advantageous when configuring convolutional layers. Notably, when combining 3D CNN, encoder, and decoder, a 3D shape can be compressed into low-dimensional latent space, and the 3D shape can be reconstructed by controlling the latent space.

We propose a method to reconstruct a 3D CAD model of mechanical parts containing machining features using a 3D encoder–decoder network that combines a 3D CNN and variational autoencoder (VAE). The main goal was to fully preserve the machining features while reconstructing the 3D CAD model using neural networks. Machining feature preserving is important in the reconstruction of mechanical parts because machining features of a 3D CAD model contain functional and manufacturing information for a part. To this end, a dataset of 3D CAD models containing fillet, chamfer, hole, and pocket machining features was built, which is generally most frequent machining features that mechanical parts contain. Then, a network architecture for reconstructing 3D voxels was proposed. This network consists of a 3D encoder compressing an input 3D voxel into a low-dimensional latent space and a 3D decoder generating a 3D voxel from the latent space. 3D encoder and decoder are configured utilizing the 3D CNN. After developing the 3D encoder–decoder network architecture, the network was trained using the training dataset, and its performance was evaluated through 3D reconstruction experiments of several test parts. By the developed 3D encoder network, the feature embedded in latent space from input voxel contains machining feature-related information. Thus, the embedded features can be used in classification task or generative task in the future.

This study makes the following academic contributions. First, a training dataset comprising 725 920 3D CAD models containing machining features was built, which can be utilized for various purposes in 3D deep learning. Second, a 3D encoder–decoder network for 3D voxels that can fully preserve the machining features was developed. In particular, the reconstruction error rate of 3D CAD models was reduced to less than 1% by applying skip connection to the 3D encoder–decoder network. In the 3D voxel reconstruction experiments, our network, compared to baseline models (McComb *et al.*, 2018; Peddireddy *et al.*, 2021), showed a notable improvement of minimum 16% to maximum 41% in reconstruction performance depending on test datasets. Third, unlike existing research where only single machining features were trained, both single and multiple machining features were simultaneously introduced and reconstructed. In addition, we verified that our proposed network can reconstruct complex 3D CAD models containing interactive features (machining features that abut or interfere with each other) through experiments. Finally, the network developed can be extended to numerous fields, including 3D model simplification, mechanical parts retrieval, and assembly component classification.

The remainder of this paper is as follows: Section 2 describes studies on 3D deep learning and its application in 3D CAD. Section 3 presents the results of the training dataset. Section 4 proposes a network for reconstructing 3D voxel models containing machining information and discusses training this network using the dataset. Section 5 discusses the results of the 3D voxel model reconstruction experiments by implementing the proposed deep learning network. Finally, Section 6 concludes the study.

2. Related Research

2.1. 3D deep learning

Prior studies on 3D deep learning can be categorized into object classification, shape segmentation, and shape reconstruction for 3D voxels, 2D images, point clouds, and meshes, which are explained below.

Object classification is a technique that recognizes the object type (e.g. chairs and furniture) corresponding to a 3D model, and previous studies related to this have presented a variety of approaches using deep neural networks (DNNs). As CNNs demonstrated remarkable performance in classifying 2D images, studies on 3D data have begun, which are categorized into multiview-based methods (Su *et al.*, 2015; Wang *et al.*, 2017a; Yu *et al.*, 2018), volumetric-based methods (Maturana & Scherer, 2015; Wu *et al.*, 2015; Riegler *et al.*, 2017; Wang *et al.*, 2017b), and point-based methods (Qi *et al.*, 2017; Zaheer *et al.*, 2017; Joseph-Rivlin *et al.*, 2018).

Shape segmentation is a technique that divides the subparts constituting the input 3D shape or semantically segments the 3D shape. Recently, deep learning models have been developed to segment 3D shapes of various representations, including volumetric grids (Riegler *et al.*, 2017; Wang *et al.*, 2017b), point clouds (Klokov & Lempitsky, 2017; Huang *et al.*, 2018; Zhang *et al.*, 2020), multiview rendering (Kalogerakis *et al.*, 2017), and surface mesh (Yi *et al.*, 2017; Wang *et al.*, 2018b). These studies primarily seek to replace traditional shape-segmentation methods with data-based learning methods.

Shape reconstruction is a technique that compresses a 3D shape into a latent space through a 3D encoder and then generates a 3D shape through a 3D decoder. Unlike traditional multiview stereo algorithms, deep learning models can encode prior knowledge of the 3D shape space that helps to resolve ambiguities in the input data (Mescheder *et al.*, 2019). Shape reconstruction techniques are classified into voxel-based reconstruction (Brock *et al.*, 2016; Sharma *et al.*, 2016; Wu *et al.*, 2016; Dai *et al.*, 2017b; Liu *et al.*, 2020), point cloud-based reconstruction (Fan *et al.*, 2017; Achlioptas *et al.*, 2018; Yu *et al.*, 2018; Song *et al.*, 2021), and mesh-based reconstruction (Ranjan *et al.*, 2018; Wang *et al.*, 2018a) based on the input data representation.

To control 3D shapes, training a deep learning model using a large-scale 3D CAD model dataset is essential. The 3D data used in deep learning consisted of real-world data and synthetic data rendered from CAD models. The former is expensive to collect and typically contains noise and occlusion, whereas, for the latter, large quantities of clean data can be generated. However, a network trained using synthetic data may have limited generalization ability.

ModelNet (Wu *et al.*, 2015) is the most commonly used dataset for 3D object classification. It comprises approximately 130 000 3D CAD models, with subsets ModelNet10 and ModelNet40. The former has 4899 models from 10 categories, and the latter has 12 311 models from 40 categories. The SUNCG (Song *et al.*, 2017) comprises approximately 400 000 full-room models. Although this dataset was built from synthetic data, it was realistically verified after comparing each model to real-world data. Dai *et al.* (2017a) created a labeled dataset collected from 3D video sequences of real indoor scenes.

2.2 3D deep learning in the field of 3D CAD

Deep learning technology utilizing 3D CAD models has diverse applications in the product development stage, such as extracting design information from boundary representation 3D models, supporting 3D design automation, production planning and monitoring, and inspection of products. Wu *et al.* (2015) converted 3D CAD models composed of irregular and unordered point clouds into standardized voxel grids and then classified them using a 3D convolution filter. Qi *et al.* (2016) combined two types of networks to classify the models. The first network has a structure that simultaneously trains all and part of the 3D model by using a multilayer perceptron convolution (MLPConv) layer, thus preventing overfitting; the second network projected 2D images from a 3D voxel using the end-to-end method, showing an effect similar to that of the multiview method.

3D CAD model retrieval is an essential technique for managing and reusing existing 3D CAD models. Wang *et al.* (2019) proposed NormalNet for 3D object recognition and the reflection–convolution–concatenation module for 3D CAD model retrieval. To retrieve 3D CAD models, Muraleedharan *et al.* (2019) first performed gauss-map-based feature extraction and then unsupervised part clustering using an autoencoder network. Zhang *et al.* (2019) proposed a new view-based 3D CAD model retrieval method. They also built a multiview dataset, MV560, from 560 3D CAD models for network training. Kim *et al.* (2020a) proposed a method to classify component types from segmented point clouds using MVCNN (Su *et al.*, 2015) and PointNet++ (Qi *et al.*, 2017) to retrieve the piping component catalogs corresponding to segmented point clouds required in reverse engineering of 3D CAD models from plant scanned point clouds. Manda *et al.* (2021b) proposed a network to retrieve the mechanical components by using 2D sketch data and 3D CAD model. They built a CADSketchNet dataset comprised of 59 514 sketch data.

Researchers have recently proposed methods for recognizing multiple machining features in a 3D model by utilizing an artificial neural network. Jian *et al.* (2018) incorporated a graph-based method with the novel bat algorithm (NBA), which was developed to complement existing neural networks having long training times, for proposing an improved NBA. Zhang *et al.* (2018) applied a 3D CNN to recognize 24 types of machining features. Shi *et al.* (2020) utilized MsvNet, a deep learning technique based on multiple sectional view (MSV) representation, for recognizing machining features. To classify machining processes in CAD models, Peddireddy *et al.* (2020) trained a network for feature recognition with a 3D CNN and then extended this network through deep transfer learning. Ghadai *et al.* (2018) applied a 3D CNN to classify manufacturability in models having drilled holes. McComb *et al.* (2018) applied a VAE to obtain unique additive manufacturing geometries. Peddireddy *et al.* (2021) applied a 3D CNN to recognize 24 types of machining features in a voxelized 3D CAD model. Some studies that classified machining features using a 3D CNN built a dataset with only single machining features (Zhang *et al.*, 2018; Peddireddy *et al.*, 2020). However, the networks trained in those studies showed low accuracy when classifying multiple machining features. Because they do not include training information for multiple machining features. The experimental results suggest that training for multiple machining features is necessary when performing reconstruction using a 3D CNN.

Most of the datasets built for training 3D deep learning models in previous studies were composed of 3D models of everyday objects. In contrast, for mechanical parts, public training datasets include ESB (Jayanti *et al.*, 2006) and DSB (Bespalov *et al.*, 2005). The former, comprising 867 3D CAD models, is a benchmark dataset built by Purdue University for comparing 3D shape-based retrieval algorithms. The latter, comprising 907 3D CAD models, is a benchmark dataset built by Drexel University to evaluate techniques for automatically classifying and retrieving CAD models. Moreover, Fusion 360 (Willis *et al.*, 2021), recently released by Autodesk, provides a set of 8625 3D CAD models. Manda *et al.* (2021a) built a CADNET dataset comprising 3317 3D CAD models by combining ESB dataset and NDR dataset. Kim *et al.* (2020b) built an MCB dataset for retrieving 3D CAD models. This dataset is comprised of 58 696 3D CAD models. However, as these datasets do not contain many 3D CAD models with machining features, the data required to train a 3D encoder–decoder network are insufficient.

Most studies except the work by McComb *et al.* (2018) using existing 3D deep learning techniques in the 3D CAD field have sought to conduct classification, retrieval, and feature recognition of 3D CAD models. However, research on the 3D reconstruction of 3D CAD models containing machining features of mechanical parts is scarce.

3. Construction of Network Training Dataset

A dataset comprising many 3D CAD models must be built to train a 3D CAD model reconstruction network. While ModelNet (Wu *et al.*, 2015) is frequently used in the 3D deep learning field, it is difficult to use ModelNet in our study because it consists of 3D models of general products. Meanwhile, though ESB (Jayanti *et al.*, 2006) and DSB (Bespalov *et al.*, 2005) provide 3D CAD models for mechanical parts, most of them do not contain machining features. Hence, in this study, training dataset containing machining feature is built upon the 3D CAD models provided in ESB (Jayanti *et al.*, 2006) and DSB (Bespalov *et al.*, 2005). Additionally, newly generated 3D CAD models are included in our training dataset that contains machining features. Accordingly, as shown in Fig. 1, the constructed training datasets comprised a total of 716 905 models, with 985 in Dataset 1 and 715 920 in Dataset 2.

To build Dataset 1, we used 3D CAD models provided in ESB (Jayanti *et al.*, 2006) and DSB (Bespalov *et al.*, 2005). However, as these 3D CAD models often do not contain machining features or the features are too small or large, they were also modified as follows. First, difficult-to-modify 3D CAD models were excluded from ESB and DSB, and a total of 985 models were selected. If the model did not contain a hole or pocket feature, this was added (Fig. 2a). If the model contained a hole or pocket, the ratio of the size of the feature to the total model size was calculated; when this value was 5% or less, the model was modified to increase the size of the feature (Fig. 2b). This was because, considering the limited resolution of voxels, the size of the processed feature had to be large enough to be clearly identified for network training. After modification, the 3D CAD models were converted into a voxel format.

When developing a neural network for reconstructing 3D CAD models containing machining features using a 3D CNN, sufficient training is necessary for both simple and complex shapes. The 3D CAD models provided in ESB and DSB represent real parts and are highly complex. As such, Dataset 1, which was built by modifying the models, contributes to training for ensuring that the proposed neural network is capable of 3D reconstruction, even for parts with complex shapes.

Regarding Dataset 2, several 3D CAD models containing machining features were automatically generated using the parametric modeling function (Immonen, 2019; Zou & Feng, 2019)

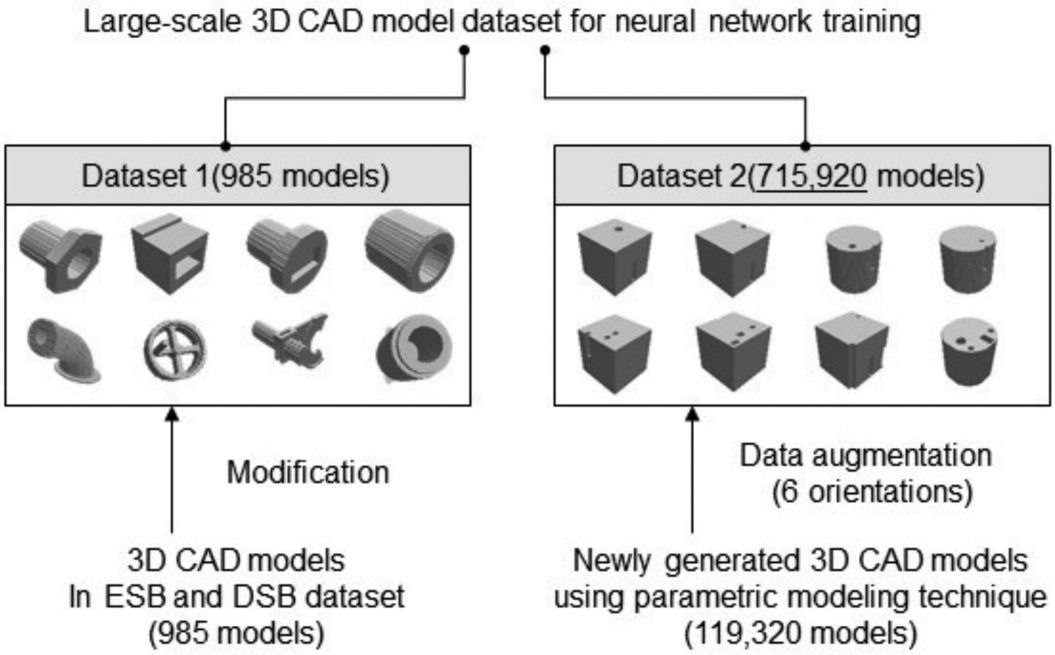

Figure 1: Composition of 3D CAD model training dataset.

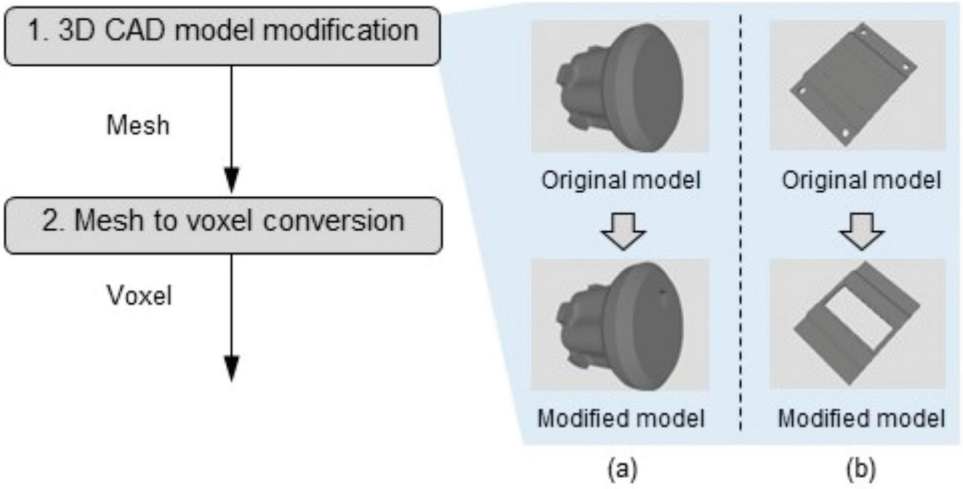

Figure 2: Construction of training data by modifying 3D CAD models of public datasets.

supported by a commercial 3D CAD system (CATIA, Dassault Systèmes), as shown in Fig. 3. Parametric modeling involves selecting the part parameters, defining equations between these parameters and 3D CAD model properties, including feature attributes, the dimensions in sketches, and constraints, and then changing the part shape by altering the parameters. This process was performed as follows. First, a template 3D CAD model containing fillet, chamfer, hole, and pocket features in a cubic or cylinder stock was generated. The parameters that control the stock size (height, width, diameter, and pad) and those controlling the feature position and size (position, diameter, height, width, and depth) were then selected, after which the relationships between these parameters and 3D CAD model properties were defined. Parametric value sets for the variants to be created were then listed in a spreadsheet, after which variant 3D CAD models were generated by applying each parameter value set using the Automation API provided in CATIA. The 3D CAD models were then converted to a voxel format.

Figure 4 shows the parameters used to control the shape of 3D CAD models. There are two stock types and ten machining feature types. Stocks are classified by cubic stock and cylinder stock. Machining feature types consist of fillet, chamfer, hole, and pocket. Hole and pocket are categorized further depending on their shapes. There are two subtypes of holes (through hole and blind hole) and six subtypes of pockets (rectangular through slot, rectangular blind slot, rectangular passage, rectangular pocket, rectangular through step, and rectangular blind step). Holes and pockets have the same parameters.

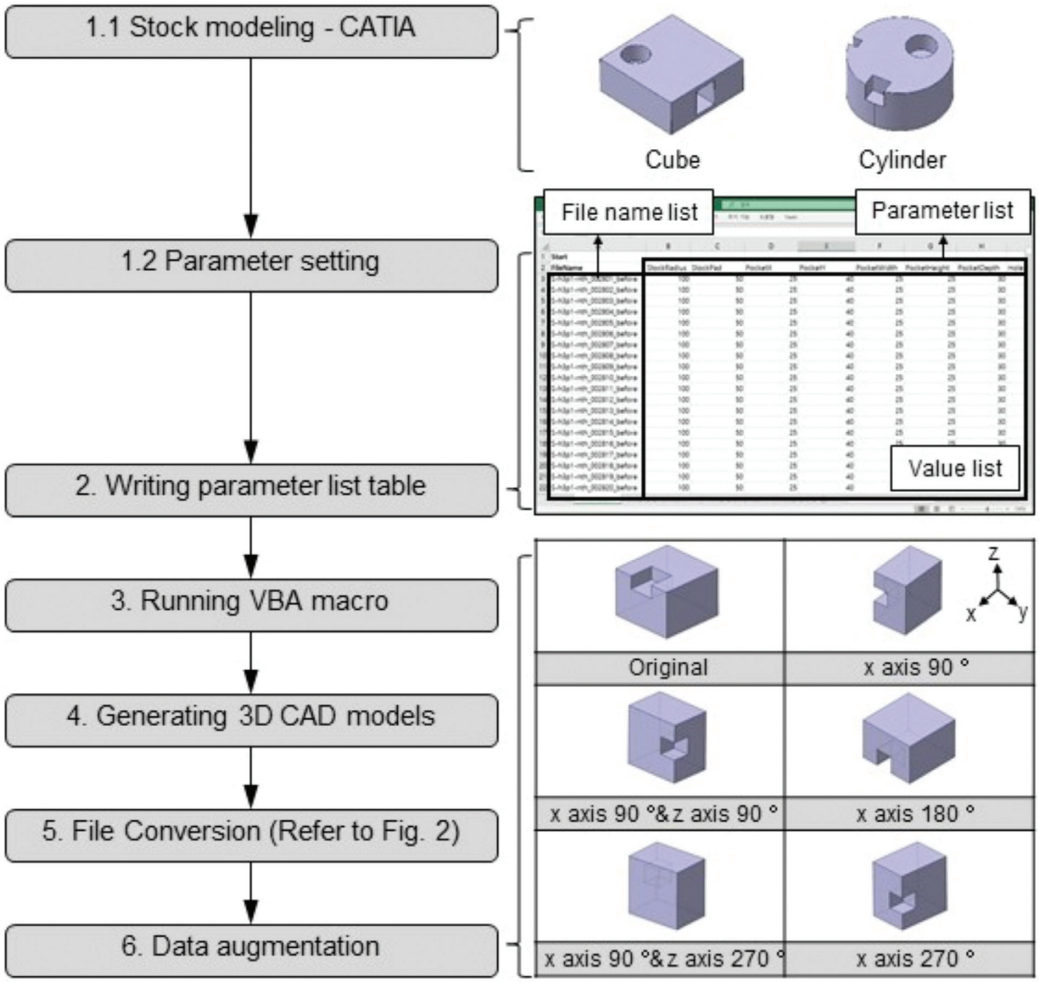

Figure 3: Construction of training data using parametric modeling function.

To demonstrate that the proposed method is applicable to multifeature CAD models, a dataset of 3D CAD models containing single feature and multifeature (multi-instance of a single feature, multi-instances of multifeature) was constructed. Feature types addressed in this study are hole, pocket, fillet, and chamfer. Multi-instance of a single feature means that multiple features of the same type are included in one model. Multi-instance of multifeature means that multiple features of different types are included in one model.

As shown in Table 1, the 3D CAD models constituting Dataset 2 are classified into single machining feature models, which contain only features of the same type, and multiple machining feature models, which contain those of different types. They are further subdivided based on the number of holes and pockets contained in the model, with a total of 10 classified cases. At least 10% of the total number of models include fillet or chamfer. A total of 119 320 3D CAD models constitute Dataset 2, which can be downloaded from the EIF lab website (Lee & Mun, 2021a).

Finally, data augmentation was applied to enhance the learning performances. 3D CAD models could be rotated along the x- or z-axis, which produces six orientations. As an example in Fig. 3, "x-axis 90° and z-axis 90°" indicates a 90° counterclockwise rotation of the model around the x-axis, followed by a 90° counterclockwise rotation around the z-axis. In the network training, we used 715 920 models as input data to the network through data augmentation. Data set comprises 336 600 single machining feature models and 379 320 multiple machining feature models. Then, these files are converted to HDF5 (HDF Group, 1997), a data format typically used for the management of large-scale data. HDF5, which manages data in a hierarchical data format, comprises data that include voxel information. To reduce the data size of the HDF file, one-bit bools were used for voxels. In addition, the chunk size was divided based on the voxel resolution, and the data was compressed using the gzip method.

The data were labeled based on the type and number of machining features contained in the 3D CAD models. This enables Dataset 2 to be used for the classification and recognition of machining features in other studies besides the reconstruction. There are two types of data-encoding methods: label encoding and one-hot encoding. In the former, the data are defined sequentially using integer values. In the latter, one is assigned to a column corresponding to a unique value, and zero is recorded in the remaining columns. Multiple labels were assigned to one piece of data in our datasets. Table 2 lists eight single machining features expressed through one-hot encoding. Multimachining features are created by combining these features. For example, a model comprising one hole and two pockets is expressed as (0, 0, 1, 0, 0, 0, 1, 0).

Figure	Type	Parameters	Range
	Cubic stock	Stock width (S_w)	[50,100]
		Stock height (S_h)	[50,100]
		Stock depth (S_d)	[50,100]
	Cylinder stock	Stock radius (S_r)	[50,100]
		Stock depth (S_d)	[20,100]
	Chamfer	Chamfer length (C_l)	[0,20]
	Fillet	Fillet radius (F_r)	[0,20]
	Through hole	Hole X coordinate (H_x)	[1, S_w-H_r]
		Hole Y coordinate (H_y)	[1, S_h-H_r]
	Blind hole	Hole radius (H_r)	[7.5, 20]

Figure	Type	Parameters	Range
	Rectangular through slot	Pocket X coordinate (P_x)	[0, P_w]
	Rectangular blind slot	Pocket Y coordinate (P_y)	[0, P_h]
	Rectangular passage	Pocket width (P_w)	[10, S_w]
	Rectangular pocket	Pocket height (P_h)	[10, S_h]
	Rectangular through step	Pocket depth (P_d)	[20, S_d]
	Rectangular blind step		

Figure 4: Parameters used to control the shape of a 3D CAD model.

Table 1: Composition of 3D CAD models in Dataset 2.

		Number of machining features		Number of files (Total: 119 320)
		Hole	Pocket	
Single machining feature models	**Case 1**	1	0	14 000
	Case 2	2	0	14 100
	Case 3	0	1	14 000
	Case 4	0	2	14 000
Multimachining feature models	**Case 5**	1	1	12 600
	Case 6	1	2	12 600
	Case 7	2	1	12 000
	Case 8	2	2	12 000
	Case 9	1	3	7020
	Case 10	3	1	7000

Table 2: One-hot label definitions for various machining features.

Index/type	Fillet	Chamfer	1 Hole	2 Holes	3 Holes	1 Pocket	2 Pockets	3 Pockets
1	1	0	0	0	0	0	0	0
2	0	1	0	0	0	0	0	0
3	0	0	1	0	0	0	0	0
4	0	0	0	1	0	0	0	0
5	0	0	0	0	1	0	0	0
6	0	0	0	0	0	1	0	0
7	0	0	0	0	0	0	1	0
8	0	0	0	0	0	0	0	1

4. 3D CAD Model Reconstruction Network

We proposed a network for the reconstruction of 3D CAD models containing machining features, which is based on a 3D encoder–decoder network and consists of encoding layers, a latent space, and decoding layers. A CNN was used to configure the encoding and decoding layers. The skip connection technique (Dai *et al.*, 2017b) is then applied to transfer the output result of each encoding layer to the corresponding decoding layer.

4.1. Model architecture

Figure 5 shows the architecture of the proposed network. Both the input and output models are in voxel format and have one

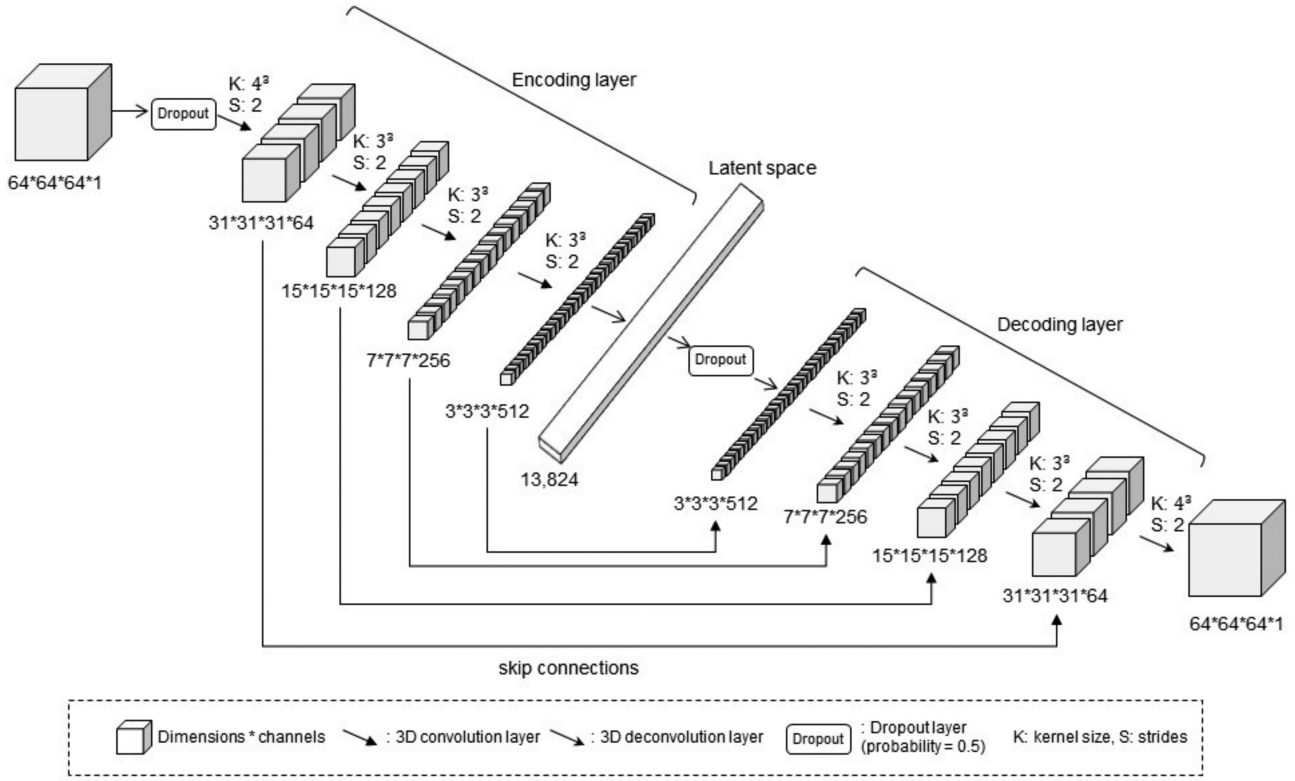

Figure 5: Architecture of 3D CAD model reconstruction neural network.

channel, representing the known space (occupied space) and unknown space (empty space). Known space is an area filled with voxels, and unknown space is an empty area that is not filled with voxels.

In the encoding layer, the input 3D shape was compressed into a latent space through 3D convolution layers. Here, instead of max pooling, downsampling was performed with a stride of 2. This is because max pooling transfers the maximum value in the area as large as the kernel size in the current layer to the next layer; as a result, the features necessary for shape reconstruction may be removed. In the decoding layer, upsampling was performed through 3D deconvolution layers based on the latent space.

3D voxels of 64 × 64 × 64 in size were input in the encoding layer. First, they were passed through a dropout layer with a probability of 0.5. Subsequently, a low-dimensional latent space was created through four 3D convolution layers. In the first convolutional layer, the kernel size was set to 4 × 4 × 4 and the stride to 2 × 2 × 2. In the second to fourth convolutional layers, these values were 3 × 3 × 3 and 2 × 2 × 2. Hence, 512 channels of 3 × 3 × 3 in size are generated through the convolutional layers. Then, to flatten the feature maps into one-dimensional vectors of length 13 824, a fully connected layer of the same length is used.

In the decoding layer, 3D models were reconstructed as the latent space containing the features of the input shapes passed through the 3D deconvolution layers as input. The kernel size, stride, and layer size followed the reverse order as that of the encoding layer. Furthermore, a skip connection similar to the U-net architecture (Ronneberger *et al.*, 2015) was added between the encoding and decoding layers. This transfers the intermediate output of the encoding layer to the decoding layer, thereby greatly enhancing the upsampling performance.

ReLU was used as an activation function for all layers forming the network. The mean squared error (MSE) was used as the loss function. Additionally, Adam (Kingma & Ba, 2015) was used as the optimizer. Section 5 describes in detail the comparison of network performances according to the loss function type, optimizer type, and use of transfer learning.

4.2. Network training

The 3D CAD model reconstruction network defined above was trained using the datasets described in Section 3. The numbers of 3D CAD models comprising the two datasets (Datasets 1 and 2) were greatly different at 985 and 715 920, respectively. As a result, if training is performed all at once, Dataset 1 will not be trained properly. To solve this problem, we trained the network in two steps, as shown in Fig. 6. First, only Dataset 1 was used for training, and the weights obtained through this were stored. Second, the weights obtained were set as the initial parameters of the network through transfer learning. The network was trained using Dataset 2. Finally, in Section 5, 3D voxel reconstruction experiments for different test cases using the trained network were shown.

Typically, transfer learning is applied to a case with small-scale data, using pre-trained weights that are trained with large-scale data in advance. However, in this study, the case where pre-trained weights are trained with Dataset 2 showed worse performance. Analysing the reason, it could be concluded that Dataset 1 has higher shape diversity and complexity and, thus, weights with high generalization characteristics were obtained

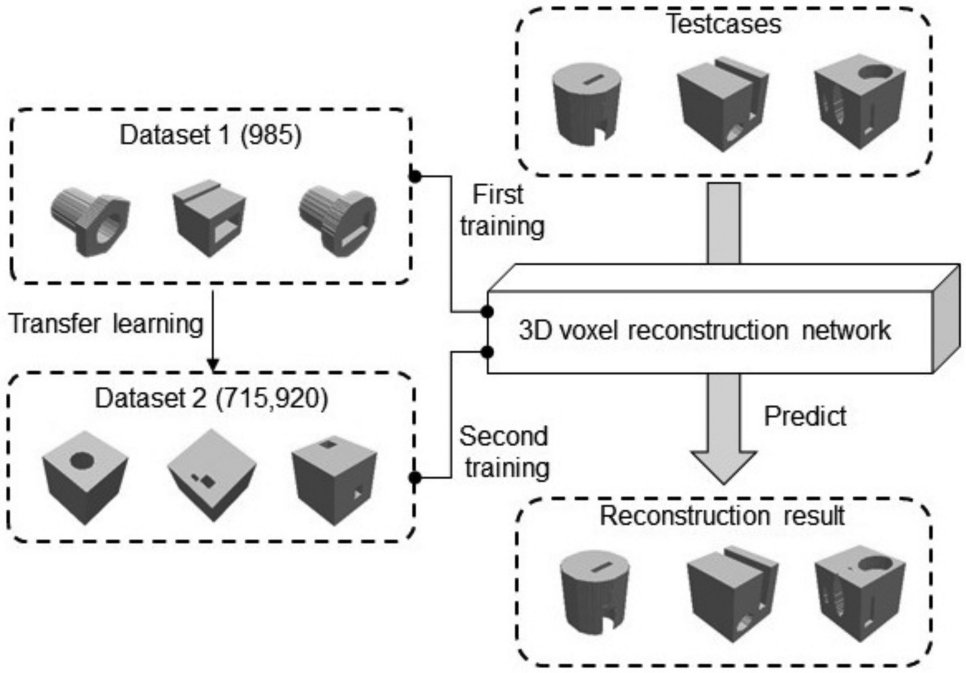

Figure 6: Network training procedure.

in the pre-learning stage even with a small number of data in Dataset 1.

5. Implementation and Experiments

Python 3.7.7 and Tensorflow 2.2.0 libraries were employed to train the deep learning model. All the experiments were conducted on a computer with an Intel Xeon E5-2640 CPU (2.4 GHz), 128 GB memory, and two Nvidia GeForce RTX 2080 Ti GPUs. A 3D CAD model reconstruction network was implemented based on the model architecture defined in Section 4. The deep learning model was implemented in Python, and the GeForce RTX 2080 Ti GPU was used for network training and prediction. The gradient descent optimization used in the experiments was Adam (Kingma & Ba, 2014). The batch size was set to eight. To prevent overfitting, the model was trained for a maximum of 150 epochs and stopped early if the validation loss did not continue to decrease in 15 epochs. The proposed network had approximately 2 billion parameters. An ablation study was also performed to demonstrate the improved performance through a comparison under different experimental conditions.

The training results were compared using different types of loss functions and optimizers to identify the influence of design choices on the deep learning model. In this experiment, Dataset 2 was used as the training data. The performance was greatly improved when using the MSE for the loss function compared to binary cross-entropy, as shown in Table 3. As for the optimizer, the performance slightly improved when using RMSProp (Tieleman & Hinton, 2012) compared to Adam.

Besides, performance improvement contributed by transfer learning was verified by using the loss curve. Figure 7 shows a graph of the training results with or without the use of transfer learning. When training from scratch, Dataset 2 was used as training data, and the weights were initialized randomly. When using transfer learning, the procedure defined in Section 4.2 was followed. In the figure, it is seen that the loss curve was

Table 3: Loss value according to type of loss function and optimizer.

Loss function	**Loss**
Binary cross-entropy	1.18e-04
MSE	1.23e-05
Optimizer	**Loss**
RMSProp (with MSE loss)	1.23e-05
Adam (with MSE loss)	1.28e-05

neither overfit nor underfit, and training was adequately performed. According to both experimental results, the model was stopped early because the performance no longer improved after 10 epochs. Comparing the validation loss values resulting from the entire training, it converged to 1.30e-05 when using only Dataset 2 and to 1.23e-05 when using transfer learning. As the converged loss values were very similar, it was not easy to compare the difference in performance. Therefore, to accurately evaluate the difference in performance, we compared the training results using an equation described in Section 5.2 that calculates the error rate of the voxels reconstructed in the 3D voxel reconstruction experiment.

5.1. Test case used in 3D voxel reconstruction experiment

Four types of datasets were used in the 3D voxel reconstruction experiments. The first dataset, named Test Dataset A, comprised 50 models selected from Dataset 1. The second, named Test Dataset B, was modeled using a CAD system and contained 50 models. The third, named Test Dataset C, comprised 50 models selected from CADNET (Manda *et al.*, 2021a) dataset. The fourth, named Test Dataset D, comprised 50 models selected from MCB (Kim *et al.*, 2020b) dataset. These models were used to evaluate the generalization ability of the trained network. Figure 8 shows the models of the datasets used in the experiment.

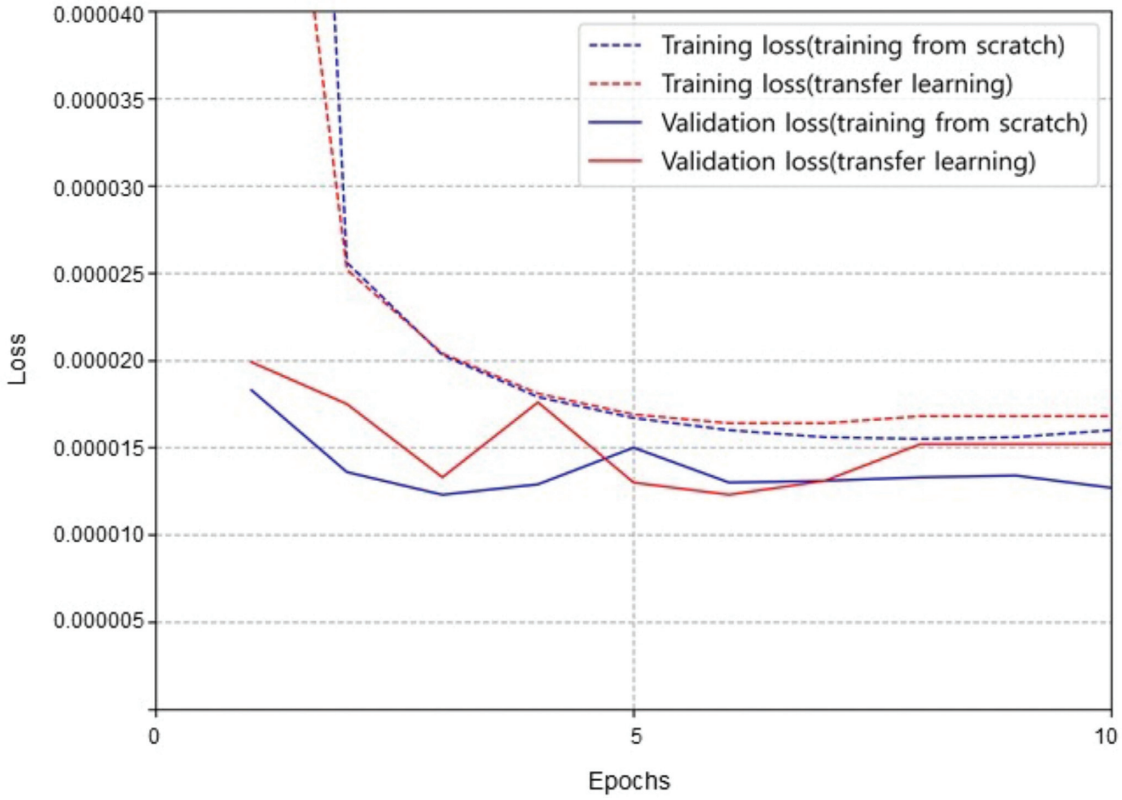

Figure 7: Loss value curve for epoch.

5.2 Ablation study

This section presents a method to calculate the average error rate of a reconstructed 3D voxel. Then, we conducted a 3D voxel reconstruction experiment and compared the average error rates of the proposed network with baseline models.

The average error rates of the voxels reconstructed in the experiment were calculated by comparing the input and output data. If the states of voxels at the same position in the input and output data are different, an error is judged to have occurred. This calculation method is possible because the voxel data are ordered. The average error rate is defined as follows:

$$\text{Average error rate}\,(\%) = \frac{100}{N}\sum_{i=1}^{N}\frac{V_i}{q^3}, \tag{1}$$

where N is the total number of test models used in the voxel reconstruction experiment, i is the test model index, q^3 is the total number of voxels in the input data, and V_i is the number of voxels recognized as errors in the ith model.

For performance comparison of the proposed network, the network by McComb *et al.* (2018) and the network by Peddireddy *et al.* (2021) were selected as baseline models. The reason for this selection was because they are the latest networks for classifying and reconstructing a voxelized 3D CAD model. In the case of Peddireddy *et al.*'s network, since this network was proposed for machining feature classification, it was adopted to define 3D encoder and 3D decoder of a VAE network, and the VAE network was used in the experiment.

Table 4 shows the average voxel error rate calculated after conducting the 3D voxel reconstruction experiment for test datasets A, B, C, and D under various experimental configurations.

Comparing all experimental results, the average voxel error rate was the lowest when both transfer learning and data augmentation were used, and our proposed network outperformed two baseline models. While Test Dataset A consists of models used for training in the network, the average error rate was calculated to be 0.51% due to their complex shapes. Meanwhile, although Test Dataset B consists of new models not used in training, the average error rate was calculated to be 0.06% due to simpler shapes. Figure 9 shows the results of the 3D voxel reconstruction experiment on the test datasets A and B in the network trained using transfer learning and data augmentation. Figure 9a and b shows the experimental results for Test Dataset A. Figure 9a shows the 10 models with the lowest error rate, while Fig. 9b shows those with the highest error rate. For Test Dataset B, Fig. 9c and d shows 10 models each with the lowest and highest error rates, respectively.

Figure 10 shows the results of the 3D voxel reconstruction experiment on the test datasets C and D in the network trained using transfer learning and data augmentation. Test datasets C and D were comprised of 3D CAD models containing interactive features from open datasets (Kim *et al.*, 2020b; Manda *et al.*, 2021a) for mechanical parts. For Test Dataset C, the average error rate was calculated to be 0.75. For Test Dataset D, the average error rate was calculated to be 0.73. According to the experimental result, we verified that our proposed network can address complex 3D CAD models containing interactive features. Figure 10 shows the results of the 3D reconstruction experiment on the test datasets C and D.

Additionally, an experiment to verify whether interactive features affect the reconstruction performance was performed. In this experiment, only Dataset 1 was used. In Dataset 1, 97 models out of 985 were 3D CAD models with interactive

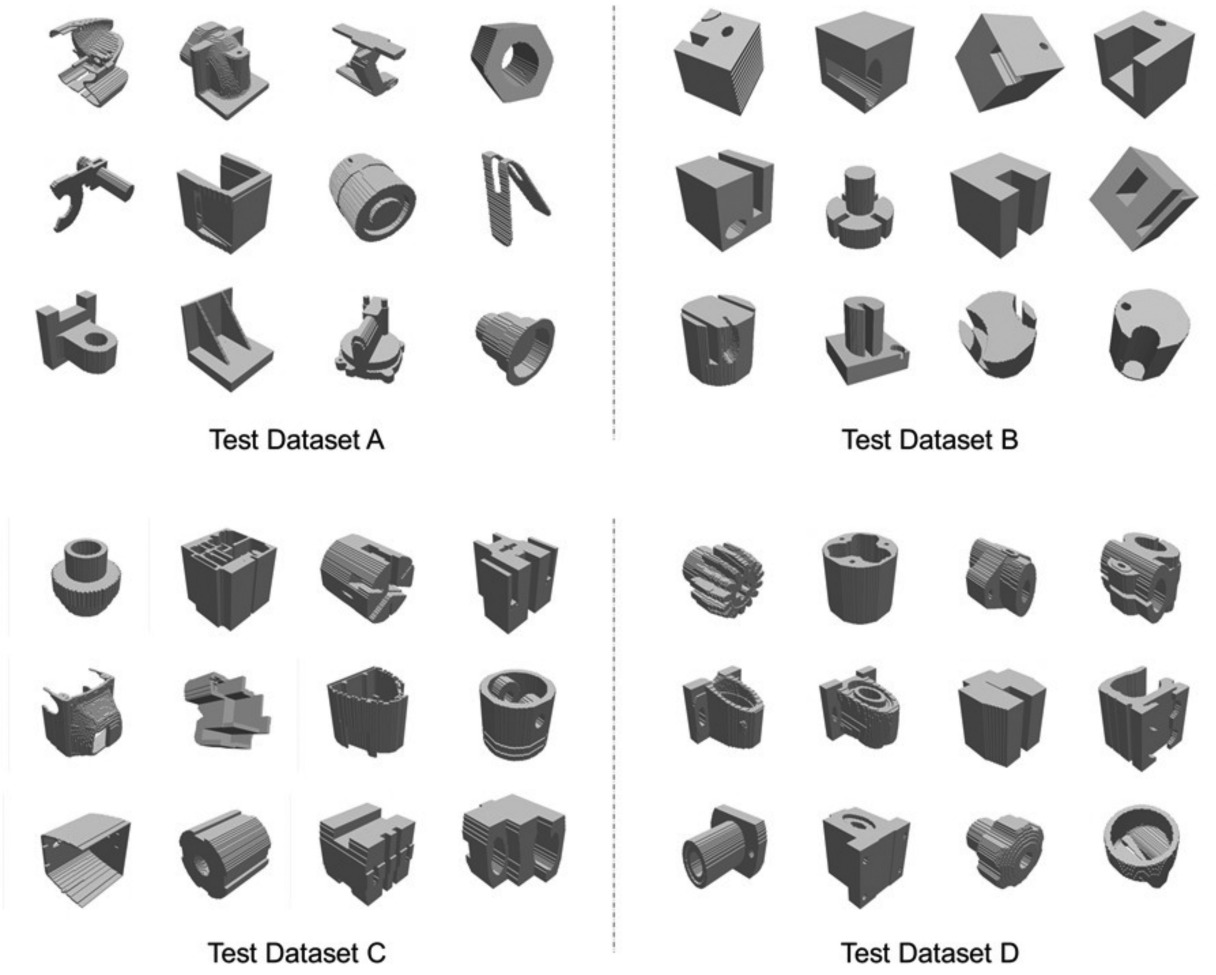

Figure 8: Composition of test datasets.

Table 4: Average voxel error rate for test cases under various experimental configurations.

Dataset	Transfer learning	Data augmentation	Average error rate (%)		
			Baseline 1 (McComb *et al.*, 2018)	Baseline 2 (Peddireddy *et al.*, 2021)	Ours
Test Dataset A			61.18	42.96	1.12
	✓		55.96	43.70	1.03
	✓	✓	42.03	28.85	**0.51**
Test Dataset B			17.93	8.91	0.11
	✓		17.58	9.66	0.09
	✓	✓	16.28	21.59	**0.06**
Test Dataset C			53.32	39.10	1.18
	✓		48.64	40.39	1.02
	✓	✓	36.81	30.97	**0.75**
Test Dataset D			55.79	43.53	1.34
	✓		46.59	44.88	1.16
	✓	✓	39.38	32.27	**0.73**

features, accounting for approximately 10%. After classifying the models with and without interactive features separately, we conducted a 3D voxel reconstruction experiment using the weights learned according to the training procedure in Section 4.2. The experiment showed that the difference in average error rate was approximately 0.1 (interactive models: 1.129 406, noninteractive models: 1.134 864); the 3D CAD models with interactive features showed a lower error rate. According to the experimental result, it was concluded that the complexity of overall shape has a more significant effect on the reconstruction than the presence or absence of interactive features.

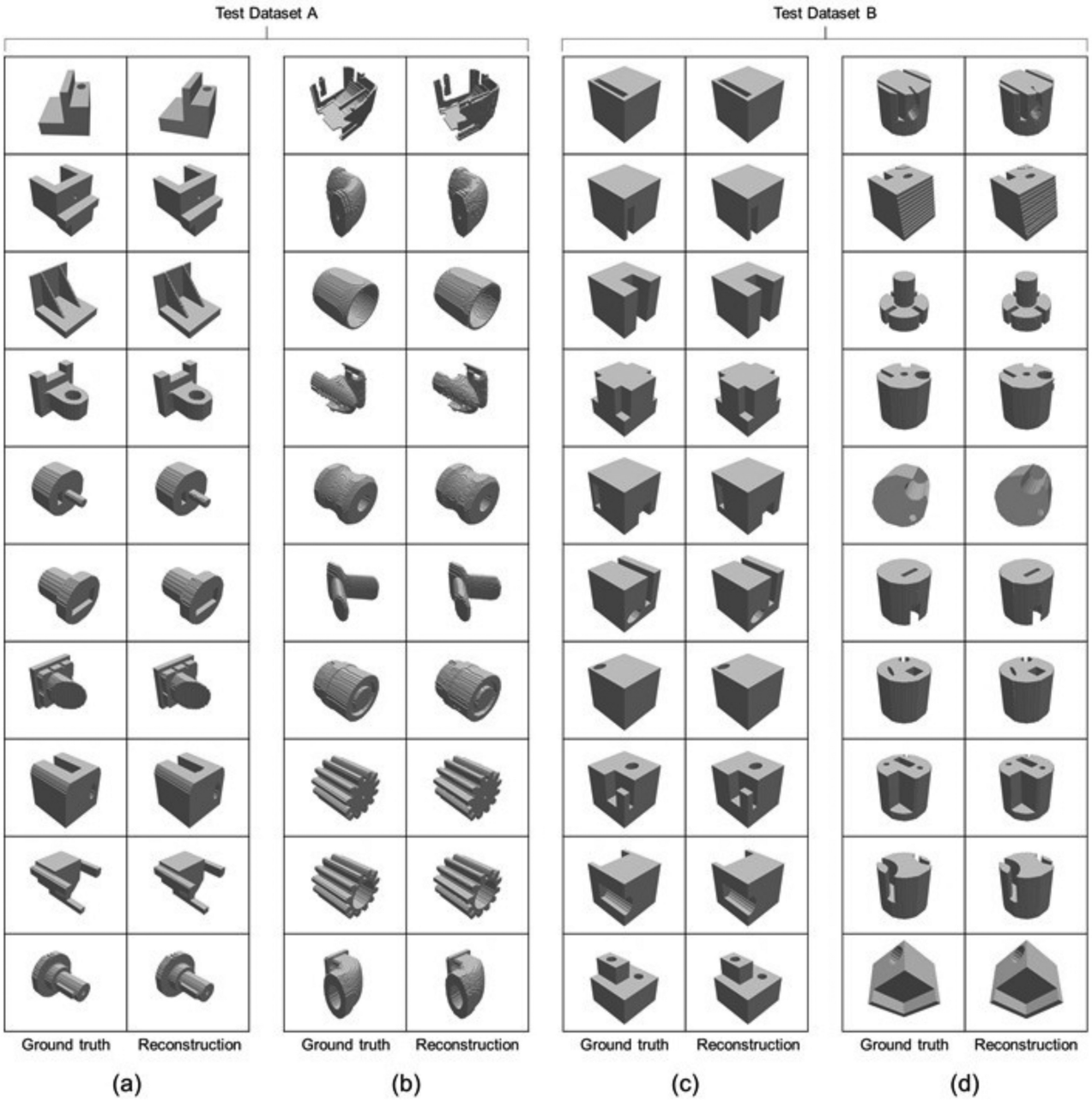

Figure 9: Results of 3D voxel reconstruction experiment for Test Dataset A and Test Dataset B.

The proposed 3D CAD model reconstruction network has the following limitations. First, the reconstructed voxels had a relatively small grid size of 64 × 64 × 64 pixels owing to the exponential increase in training time and GPU memory usage as the voxel size increased. Second, although we constructed a large-scale 3D CAD model dataset containing machining features using a parametric modeling function, the data consist of relatively simple models compared to the shapes of real mechanical parts.

This study was conducted on 3D CAD models containing machining features, and improved the network by applying skip connection to VAE. Our network showed a high reconstruction accuracy of 99% for 64 × 64 × 64 voxel resolution. In the case of shape reconstruction, it is difficult to compare with previous studies because the reconstruction accuracy can change depending on 3D CAD models, experimental conditions, and error calculation method. Hence, we selected two baseline models (McComb *et al.*, 2018; Peddireddy *et al.*, 2021) using the same 3D CAD models under the same experimental conditions, and compared 3D reconstruction performance with our proposed network. Our network, compared to baseline models, showed a notable improvement of minimum 16% to maximum 41% in reconstruction performance depending on test datasets.

Through these processes, our proposed network showed high reconstruction ability for 3D CAD models containing machining features, compared to previous studies. Our proposed network can be applied for various applications, including 3D object reconstruction from 2D images, 3D object retrieval, 3D object generation, shape completion, shape synthesis, and

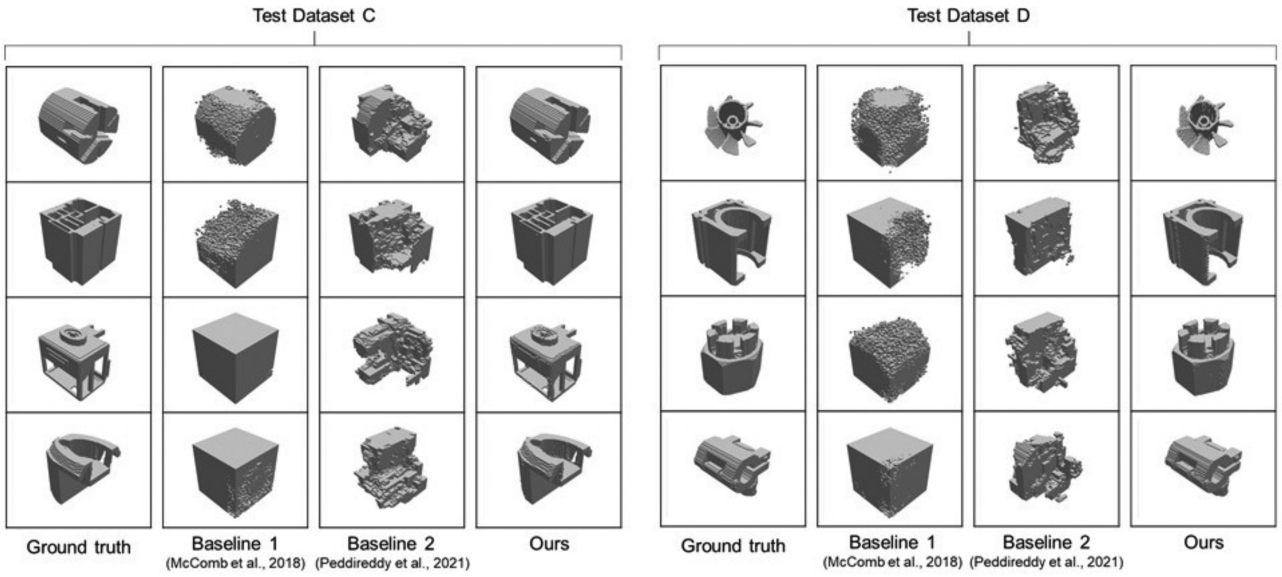

Figure 10: Results of 3D voxel reconstruction experiment for Test Dataset C and Test Dataset D.

3D model simplification (Lee *et al.*, 2020, 2021b; Ghorbani & Khameneifar, 2021; Kim *et al.*, 2021). For example, it can be integrated with a generative adversarial network (GAN) model (Goodfellow *et al.*, 2014) to develop a deep learning model that generates simplified shapes by transforming input 3D voxels.

6. Conclusions

We proposed a network that reconstructs voxel-based 3D CAD models containing machining features using a 3D CNN. To train this network, 3D CAD model datasets containing machining features were built and converted into voxel format. Furthermore, transfer learning was used to efficiently train the two datasets, which had different numbers of models and shape complexity. The proposed network demonstrated high reconstruction performance with an error rate of approximately 1%, even for new 3D CAD models generated for experimentation.

In the future, to improve the proposed 3D encoder–decoder network, more 3D CAD models used in the field will be included in the training dataset. Moreover, the processable voxel grid size will be increased by reducing the weight of the DNN. The metric used to calculate the error rate of a reconstructed 3D model needs to be improved so that it presents how well portions of interest (i.e. edges and faces comprising machining features) of a 3D model are reconstructed. In the future, we will perform research on configuring a GAN to transform and simplify 3D CAD models. Here, the proposed 3D encoder–decoder network will be used as a generative model in a GAN.

Acknowledgments

This research was supported by the AI-based gasoil plant O&M Core Technology Development Program (Project ID: 21ATOG-C161932-01) funded by the Korean government (MOLIT) and by the Basic Science Research Program (Project ID: NRF-2019R1F1A1053542&NRF-2021R1I1A1A01050440) through the National Research Foundation of Korea (NRF) funded by the Korean government (MSIT) and by a Korea University Grant 2020.

Conflict of interest statement

None declared.

References

Achlioptas, P., Diamanti, O., Mitliagkas, I., & Guibas, L. J. (2018). Learning representations and generative models for 3D point clouds. In *Proceedings of the International Conference on Machine Learning*.

Bespalov, D., Ip, C. Y., Regli, W. C., & Shaffer, J. (2005). Benchmarking CAD search techniques. In *Proceedings of the 2005 ACM Symposium on Solid and Physical Modeling*(pp. 275–286).

Brock, A., Lim, T., Ritchie, J. M., & Weston, N. (2016). Generative and discriminative voxel modeling with convolutional neural networks. In *Advances in Neural Information Processing Systems*.

Chen, J., Kira, Z., & Cho, Y. K. (2019). Deep learning approach to point cloud scene understanding for automated scan to 3D reconstruction. *Journal of Computing in Civil Engineering*, *33*(4), 04019027.

Dai, A., Chang, A. X., Savva, M., Halber, M., Funkhouser, T., & Nießner, M. (2017a). ScanNet: Richly-annotated 3d reconstructions of indoor scenes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*(pp. 5828–5839).

Dai, A., Ruizhongtai Qi, C., & Nießner, M. (2017b). Shape completion using 3d-encoder-predictor CNNs and shape synthesis. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*(pp. 5868–5877).

Eltner, A., Kaiser, A., Castillo, C., Rock, G., Neugirg, F., & Abellán, A. (2016). Image-based surface reconstruction in geomorphometry–merits, limits and developments. *Earth Surface Dynamics*, *4*(2), 359–389.

Fan, H., Su, H., & Guibas, L. J. (2017). A point set generation network for 3D object reconstruction from a single image. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.

Fu, K., Peng, J., He, Q., & Zhang, H. (2021). Single image 3D object reconstruction based on deep learning: A review. *Multimedia Tools and Applications*, *80*(1), 463–498.

Ge, L., Ren, Z., & Yuan, J. (2018). Point-to-point regression pointnet for 3d hand pose estimation. In *Lecture Notes in Computer Science. Proceedings of the European Conference on Computer Vision*(pp. 489–505).

Ghadai, S., Balu, A., Sarkar, S., & Krishnamurthy, A. (2018). Learning localized features in 3D CAD models for manufacturability analysis of drilled holes. *Computer Aided Geometric Design*, *62*, 263–275.

Ghorbani, H., & Khameneifar, F. (2021). Airfoil profile reconstruction from unorganized noisy point cloud data. *Journal of Computational Design and Engineering*, *8*(2), 740–755.

Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial networks. In *Advances in Neural Information Processing Systems*.

HDF Group. (1997). Hierarchical data format version 5. Available at: http://www.hdfgroup.org/HDF5. Accessed November 2020.

Huang, Q., Wang, W., & Neumann, U. (2018). Recurrent slice networks for 3d segmentation of point clouds. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*(pp. 2626–2635).

Immonen, E. (2019). A parametric morphing method for generating structured meshes for marine free surface flow applications with plane symmetry. *Journal of Computational Design and Engineering*, *6*(3), 348–353.

Jayanti, S., Kalyanaraman, Y., Iyer, N., & Ramani, K. (2006). Developing an engineering shape benchmark for CAD models. *Computer-Aided Design*, *38*(9), 939–953.

Jian, C., Li, M., Qiu, K., & Zhang, M. (2018). An improved NBA-based STEP design intention feature recognition. *Future Generation Computer Systems*, *88*, 357–362.

Joseph-Rivlin, M., Zvirin, A., & Kimmel, R. (2018). Mo-Net: Flavor the moments in learning to classify shapes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*.

Kalogerakis, E., Averkiou, M., Maji, S., & Chaudhuri, S. (2017). 3D shape segmentation with projective convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*(pp. 3779–3788).

Kim, H., Yeo, C., Lee, I. D., & Mun, D. (2020a). Deep-learning-based retrieval of piping component catalogs for plant 3D CAD model reconstruction. *Computers in Industry*, *123*, 103320.

Kim, S., Chi, H. G., Hu, X., Huang, Q., & Ramani, K. (2020b). A large-scale annotated mechanical components benchmark for classification and retrieval tasks with deep neural networks. In *Proceedings of the European Conference on Computer Vision*.

Kim, B. C., Lee, H., Mun, D., & Han, S. (2021). Lifecycle management of component catalogs based on a neutral model to support seamless integration with plant 3D design. *Journal of Computational Design and Engineering*, *8*(1), 409–427.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations*.

Klokov, R., & Lempitsky, V. (2017). Escape from cells: Deep kd-networks for the recognition of 3d point cloud models. In *Proceedings of the IEEE International Conference on Computer Vision*(pp. 863–872).

Lee, W. H., Lee, K. H., Lee, J. M., & Nam, B. W. (2020). Registration method for maintenance-work support based on augmented-reality-model generation from drawing data. *Journal of Computational Design and Engineering*, *7*(6), 775–787.

Lee, H., & Mun, D., Dataset of 3D CAD models containing machining features in OBJ format. (2021a). http://www.dhmun.net/home/Research_Data. Retrieved March 2021.

Lee, I. D., Lee, I., & Han, S. (2021b). 3D reconstruction of as-built model of plant piping system from point clouds and port information. *Journal of Computational Design and Engineering*, *8*(1), 195–209.

Liu, J., Mills, S., & McCane, B. (2020). Variational Autoencoder for 3D Voxel Compression. In *2020 35th International Conference on Image and Vision Computing New Zealand*(pp. 1–6).

Manda, B., Bhaskare, P., & Muthuganapathy, R. (2021a). A convolutional neural network approach to the classification of engineering models. *IEEE Access*, *9*, 22711–22723.

Manda, B., Dhayarkar, S., Mitheran, S., Viekash, V. K., & Muthuganapathy, R. (2021b). 'CADSketchNet'-An annotated sketch dataset for 3D CAD model retrieval with deep neural networks. *Computers and Graphics*, *99*, 100–113.

Maturana, D., & Scherer, S. (2015). VoxNet: A 3D convolutional neural network for real-time object recognition. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*(pp. 922–928).

McComb, C., Meisel, N., Murphy, C., & Simpson, T. W. (2018). Predicting part mass, required support material, and build time via autoencoded voxel patterns. In *29th Annual International Solid Freeform Fabrication Symposium*(pp. 1–15).

Mescheder, L., Oechsle, M., Niemeyer, M., Nowozin, S., & Geiger, A. (2019). Occupancy networks: Learning 3d reconstruction in function space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*(pp. 4460–4470).

Muraleedharan, L. P., Kannan, S. S., & Muthuganapathy, R. (2019). Autoencoder-based part clustering for part-in-whole retrieval of CAD models. *Computers and Graphics*, *81*, 41–51.

Peddireddy, D., Fu, X., Wang, H., Joung, B. G., Aggarwal, V., Sutherland, J. W., & Jun, M. B. G. (2020). Deep learning-based approach for identifying conventional machining processes from CAD data. *Procedia Manufacturing*, *48*, 915–925.

Peddireddy, D., Fu, X., Shankar, A., Wang, H., Joung, B. G., Aggarwal, V., Sutherland, J. W., & Jun, M. B. G. (2021). Identifying manufacturability and machining processes using deep 3D convolutional networks. *Journal of Manufacturing Processes*, *64*, 1336–1348.

Qi, C. R., Su, H., Nießner, M., Dai, A., Yan, M., & Guibas, L. J. (2016). Volumetric and multi-view CNNs for object classification on 3d data. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*(pp. 5648–5656).

Qi, C. R., Yi, L., Su, H., & Guibas, L. J. (2017). PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Conference on Neural Information Processing Systems (NIPS)*.

Ranjan, A., Bolkart, T., Sanyal, S., & Black, M. J. (2018). Generating 3D faces using convolutional mesh autoencoders. In *Proceedings of the European Conference on Computer Vision*(pp. 725–741).

Riegler, G., Ulusoy, A. O., & Geiger, A. (2017). OctNet: Learning deep 3D representations at high resolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*(pp. 3577–3586).

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*(pp. 234–241). Springer.

Sharma, A., Grau, O., & Fritz, M. (2016). Vconv-dae: Deep volumetric shape learning without object labels. In *European Conference on Computer Vision*(pp. 236–250). Springer.

Shi, P., Qi, Q., Qin, Y., Scott, P. J., & Jiang, X. (2020). A novel learning-based feature recognition method using multiple

sectional view representation. *Journal of Intelligent Manufacturing*, *31*(5), 1291–1309.

Song, S., Yu, F., Zeng, A., Chang, A. X., Savva, M., & Funkhouser, T. (2017). Semantic scene completion from a single depth image. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*(pp. 1746–1754).

Song, J., Lee, J., Ko, K., Kim, W. D., Kang, T. W., Kim, J. Y., & Nam, J. H. (2021). Unorganized point classification for robust NURBS surface reconstruction using a point-based neural network. *Journal of Computational Design and Engineering*, *8*(1), 392–408.

Su, H., Maji, S., Kalogerakis, E., & Learned-Miller, E. (2015). Multi-view convolutional neural networks for 3d shape recognition. In *Proceedings of the IEEE International Conference on Computer Vision*(pp. 945–953).

Yu, T., Meng, J., & Yuan, J. (2018). Multi-view harmonized bilinear network for 3d object recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*(pp. 186–194).

Tieleman, T., & Hinton, G. (2012). Lecture 6.5–RmsProp: Divide the gradient by a running average of its recent magnitude. In *COURSERA: Neural networks for machine learning*.

Tulsiani, S., Zhou, T., Efros, A. A., & Malik, J. (2017). Multi-view supervision for single-view reconstruction via differentiable ray consistency. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*(pp. 2626–2634).

Wang, C., Pelillo, M., & Siddiqi, K. (2017a). Dominant set clustering and pooling for multi-view 3d object recognition. In *Proceedings of the British Machine Vision Conference*.

Wang, P.-S., Liu, Y., Guo, Y.-X., Sun, C.-Y., & Tong, X. (2017b). O-CNN: Octree-based convolutional neural networks for 3d shape analysis. *ACM Transactions on Graphics*, *36*(4), 1–11.

Wang, N., Zhang, Y., Li, Z., Fu, Y., Liu, W., & Jiang, Y.-G. (2018a). Pixel2Mesh: Generating 3D mesh models from single RGB images. In *Proceedings of the European Conference on Computer Vision*(pp. 52–67).

Wang, P., Gan, Y., Shui, P., Yu, F., Zhang, Y., Chen, S., & Sun, Z. (2018b). 3d shape segmentation via shape fully convolutional networks. *Computers and Graphics*, *70*, 128–139.

Wang, C., Cheng, M., Sohel, F., Bennamoun, M., & Li, J. (2019). NormalNet: A voxel-based CNN for 3D object classification and retrieval. *Neurocomputing*, *323*, 139–147.

Willis, K. D., Pu, Y., Luo, J., Chu, H., Du, T., Lambourne, J. G., Solar-Lezama, A., & Matusik, W. (2021). Fusion 360 gallery: A dataset and environment for programmatic CAD reconstruction.In *International Conference on Learning Representations*.

Wu, Z., Song, S., Khosla, A., Yu, F., Zhang, L., Tang, X., & Xiao, J. (2015). 3D ShapeNets: A deep representation for volumetric shapes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*(pp. 1912–1920).

Wu, J., Zhang, C., Xue, T., Freeman, B., & Tenenbaum, J. (2016). Learning a probabilistic latent space of object shapes via 3D generative-adversarial modeling. In *Advances in Neural Information Processing Systems*.

Xu, Q., Wang, W., Ceylan, D., Mech, R., & Neumann, U. (2019). DISN: Deep implicit surface network for high-quality single-view 3d reconstruction. In *Advances in Neural Information Processing Systems*.

Yi, L., Su, H., Guo, X., & Guibas, L. J. (2017). SyncSpecCNN: Synchronized spectral CNN for 3d shape segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*(pp. 6584–6592).

Yu, L., Li, X., Fu, C. W., Cohen-Or, D., & Heng, P. A. (2018). Pu-net: Point cloud upsampling network. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*(pp. 2790–2799).

Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., & Smola, A. J. (2017). Deep sets. In *Conference on Neural Information Processing Systems (NIPS)*. (pp. 3391–3401)

Zhang, Z., Jaiswal, P., & Rai, R. (2018). Featurenet: Machining feature recognition based on 3d convolution neural network. *Computer-Aided Design*, *101*, 12–22.

Zhang, C., Zhou, G., Yang, H., Xiao, Z., & Yang, X. (2019). View-based 3d CAD model retrieval with deep residual networks. *IEEE Transactions on Industrial Informatics*, *16*(4), 2335–2345.

Zhang, D., He, F., Tu, Z., Zou, L., & Chen, Y. (2020). Point-wise geometric and semantic learning network on 3D point clouds. *Integrated Computer-Aided Engineering*, *27*(1), 57–75.

Zou, Q., & Feng, H. Y. (2019). Variational B-rep model analysis for direct modeling using geometric perturbation. *Journal of Computational Design and Engineering*, *6*(4), 606–616.